



Audio Computing Lab

Microphone Arrays

Paris Smaragdis
paris@illinois.edu
paris.cs.illinois.edu

Overview

- **Microphone arrays**
 - What are they?
 - What can they do?
- **Beamforming**
 - Focusing on a direction
 - Ignoring a direction
- **Localization**
 - GCC-PHAT
 - Energy methods

Remember the advantage of two ears?



The “spatial” dimension

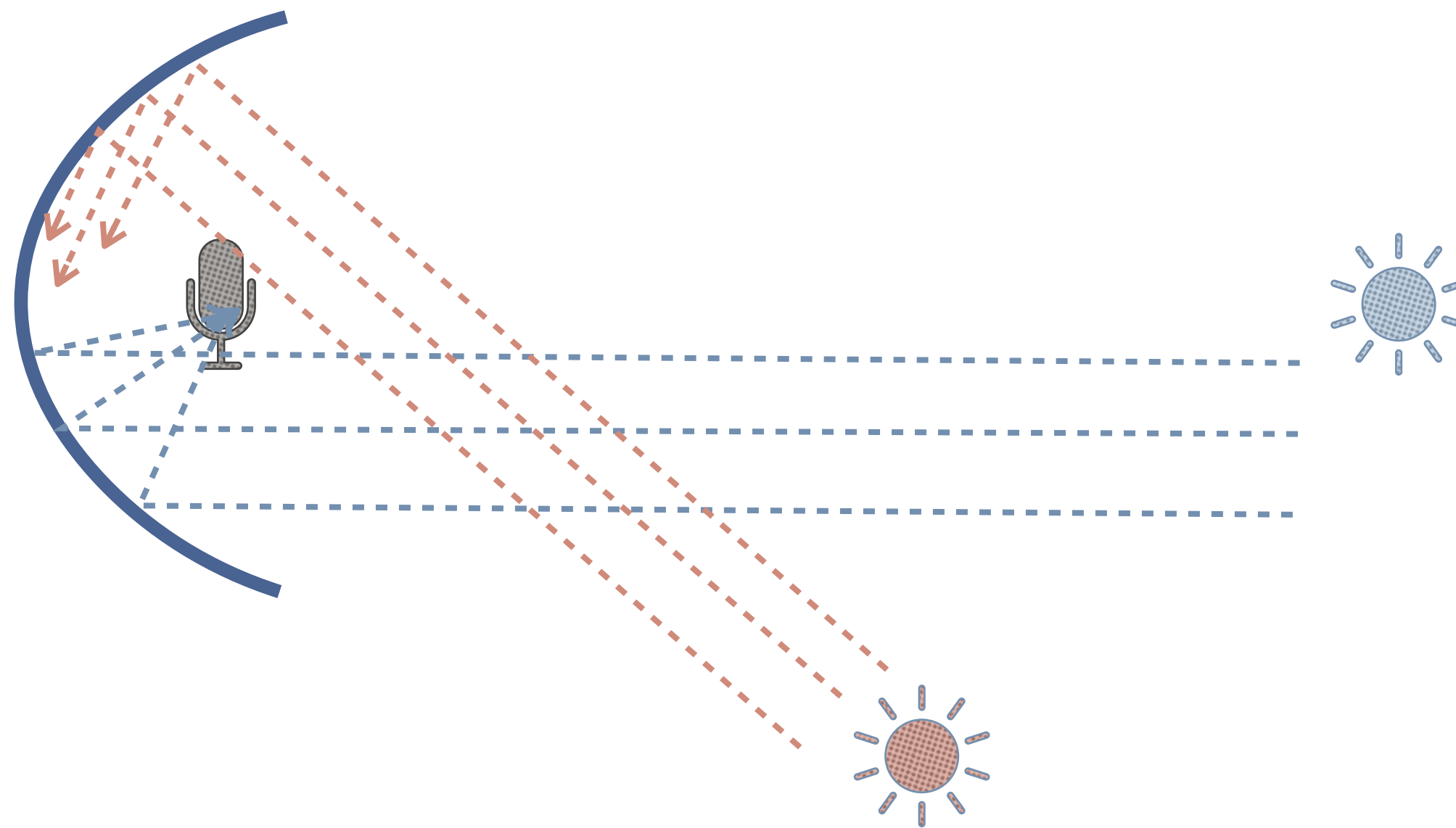
- With two ears (and eyes) we perceive in 3D
 - This reveals the *spatial* location of an object
- Allows for two important functions
 - Focusing on a desired direction
 - Locating where a sound is coming from

Why are our ears shaped like this?



Focusing towards a direction

- Parabolic dishes
 - Redirect all sound coming from straight ahead, to a sensor



You might have seen that before

- Especially if you are a sports fan

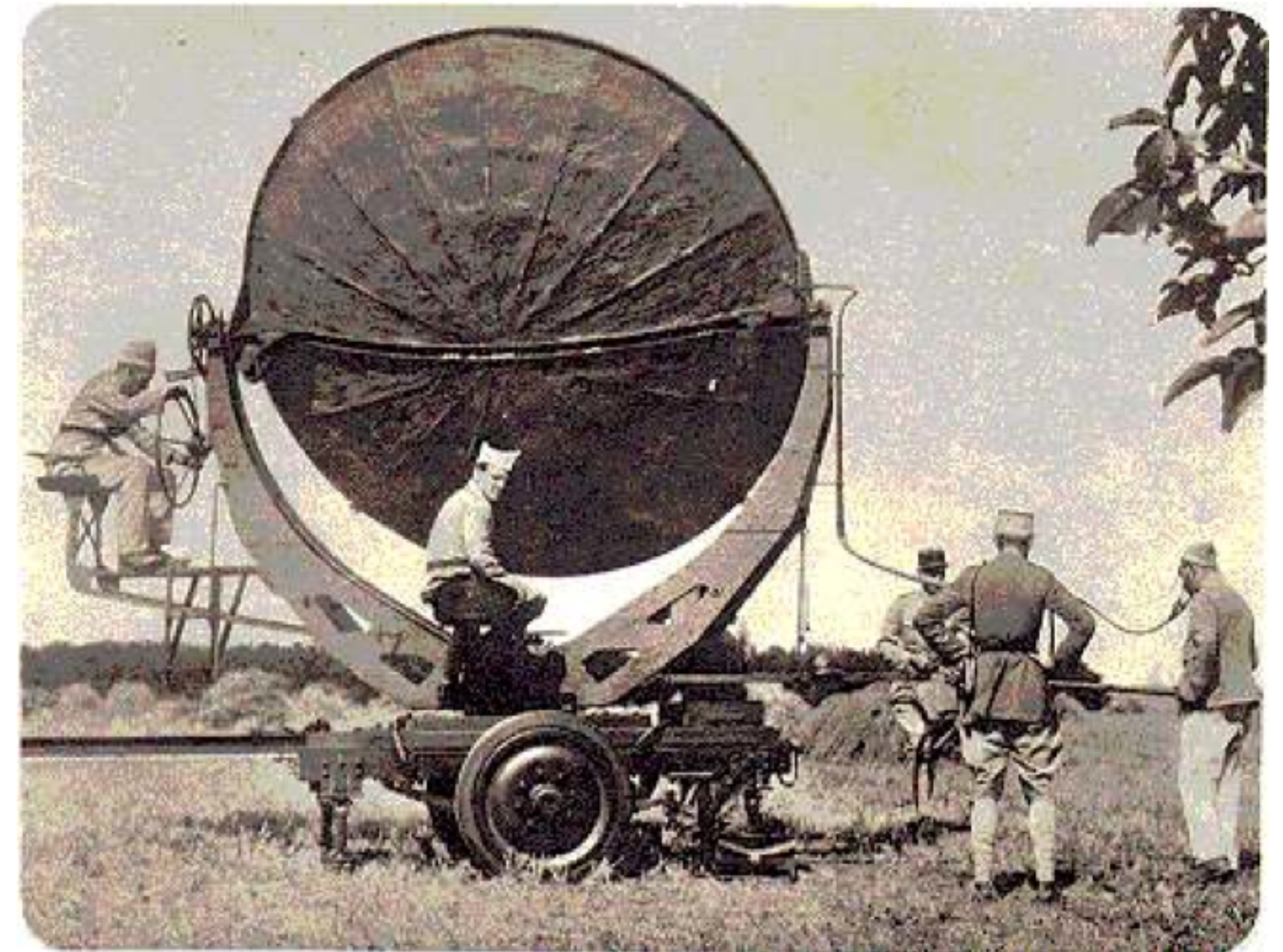


Which is an old idea

- What we used before radar

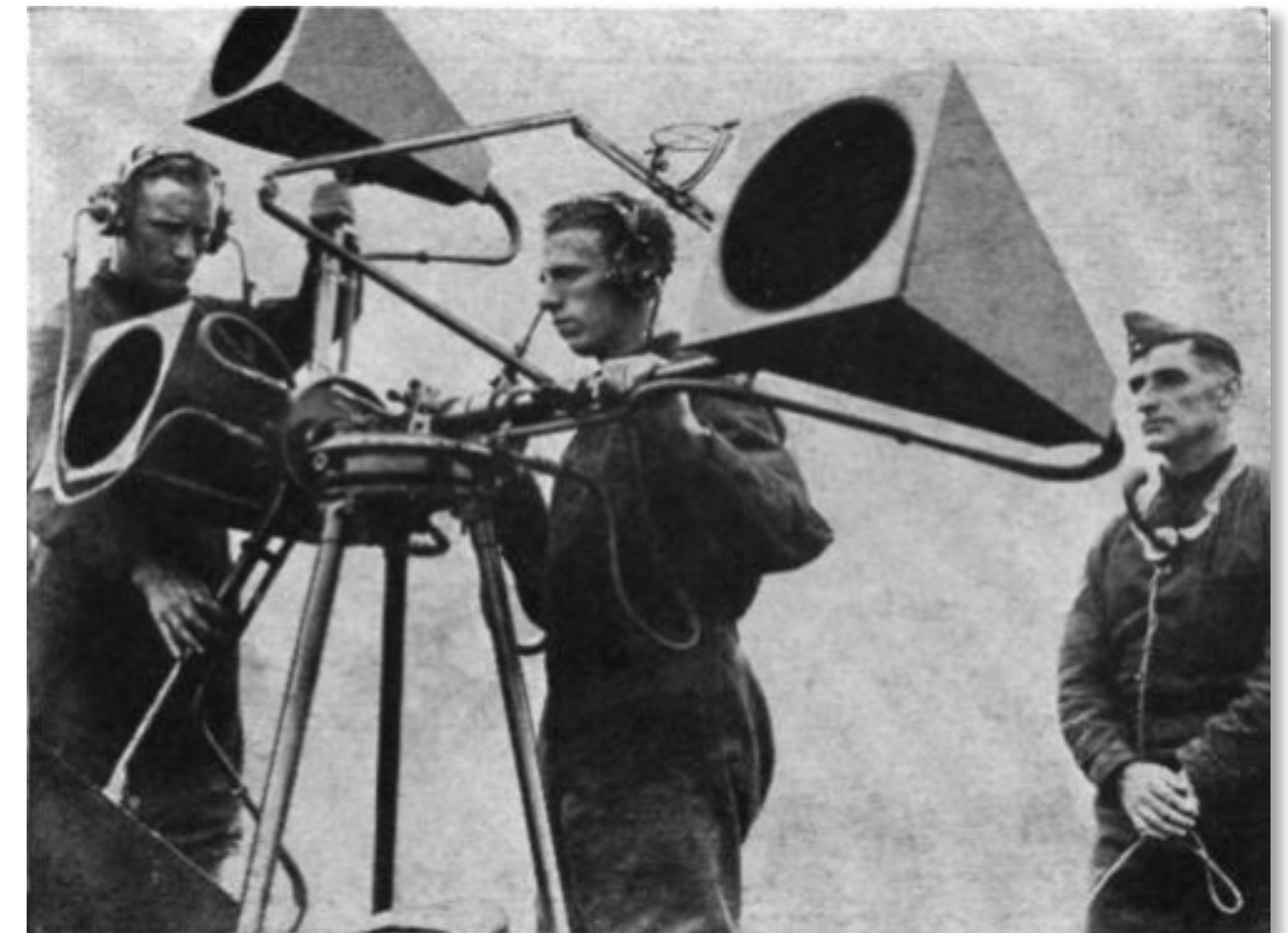


WW1 "Acoustic Mirror"



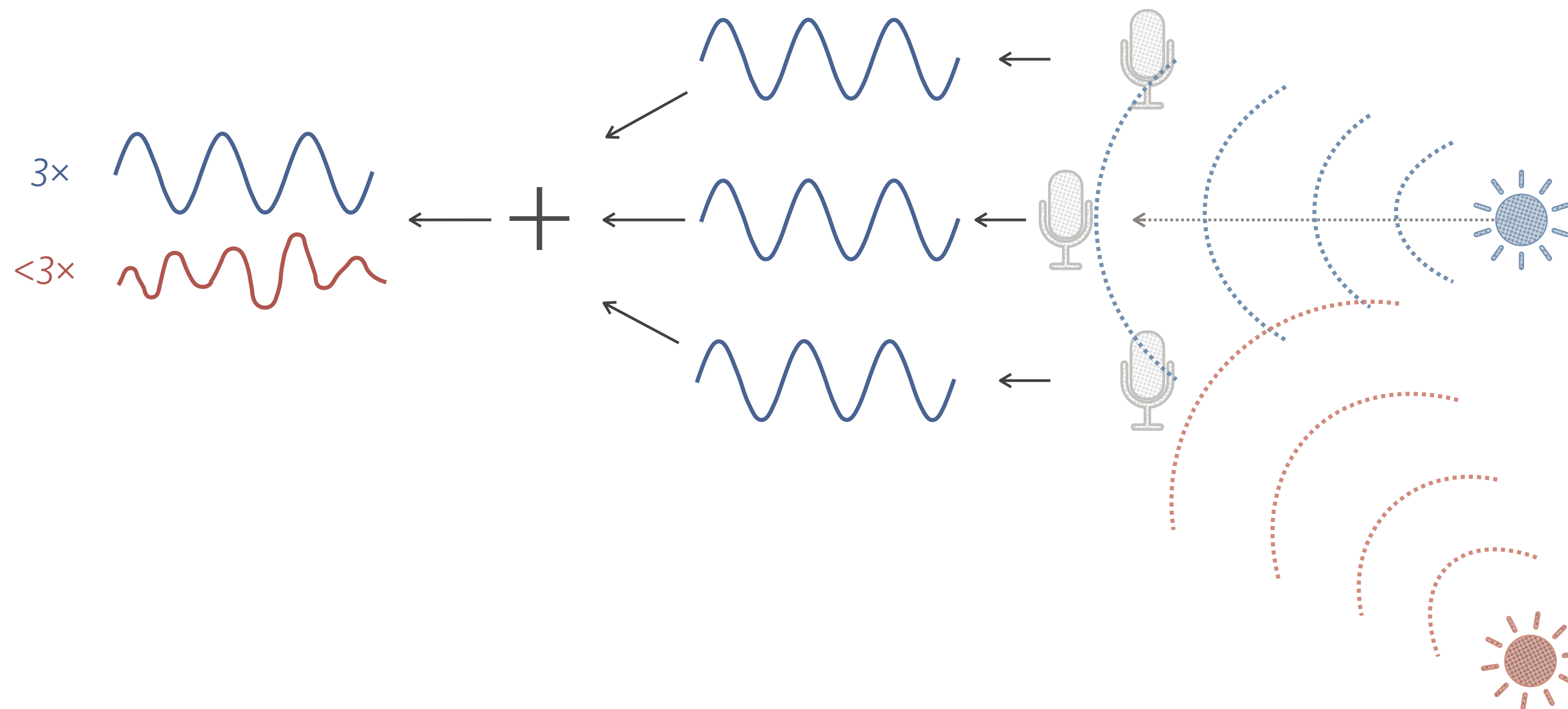
1936 parabolic detector

An idea with weird manifestations ...



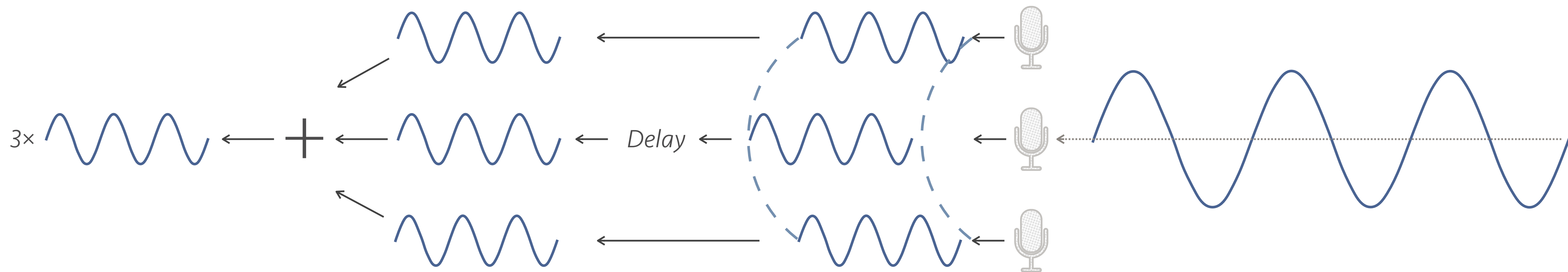
Discretizing the parabolic dish

- Instead of a continuous surface we can use a small set of microphones to emulate it
 - What's the practical problem here here?



A more flexible approach

- We can instead move microphones virtually using time delays on the recorded signals



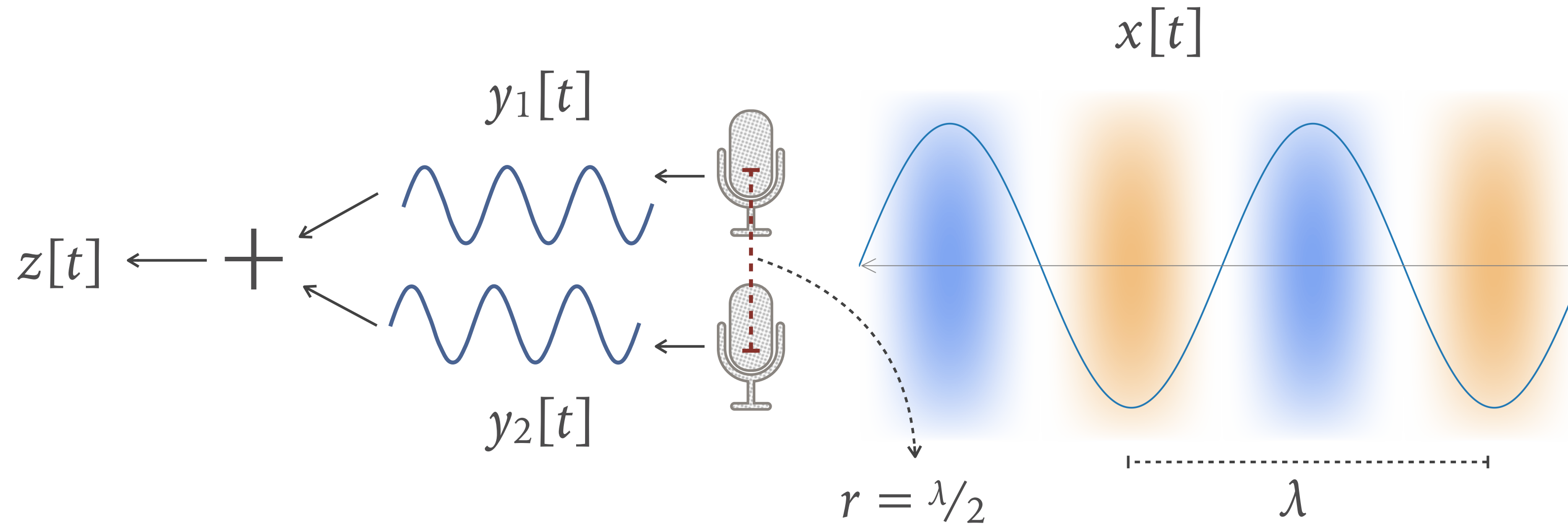
*How do we delay?
Convolve with $[0, 0, \dots, 0, 1]$
(or a shifted sinc)*

Formalizing the process

- Instead of a delay we will use filters
 - Remember that a filter can also just implement a delay
 - But it can do a lot more as well
- We will work in the frequency domain
 - Makes the math a little cleaner
- We will start with a single frequency

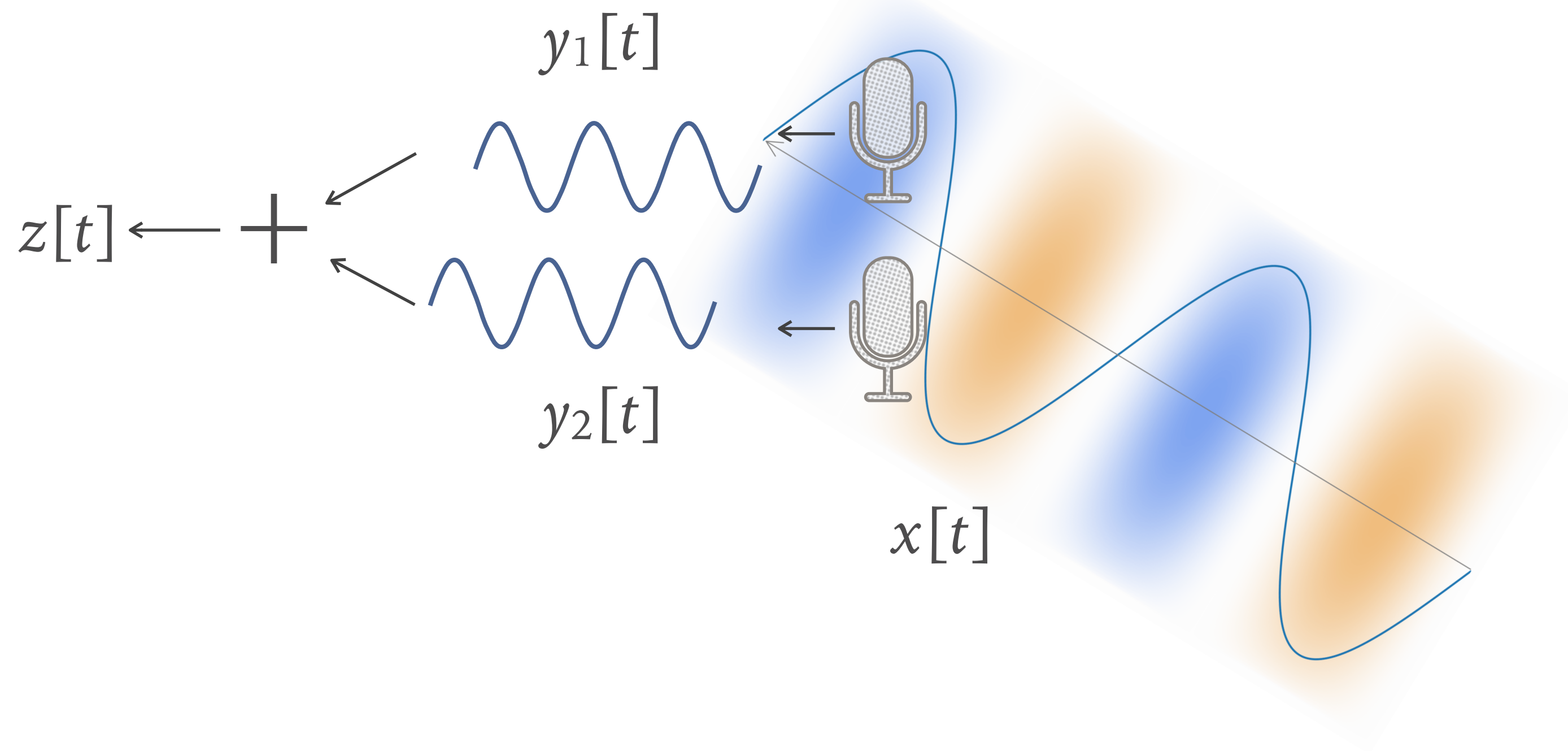
Two mics, one sine

- We record a sine wave with two mics
 - Distance between the mics is sine's wavelength $\lambda/2$
 - What happens when the sine comes from the front?



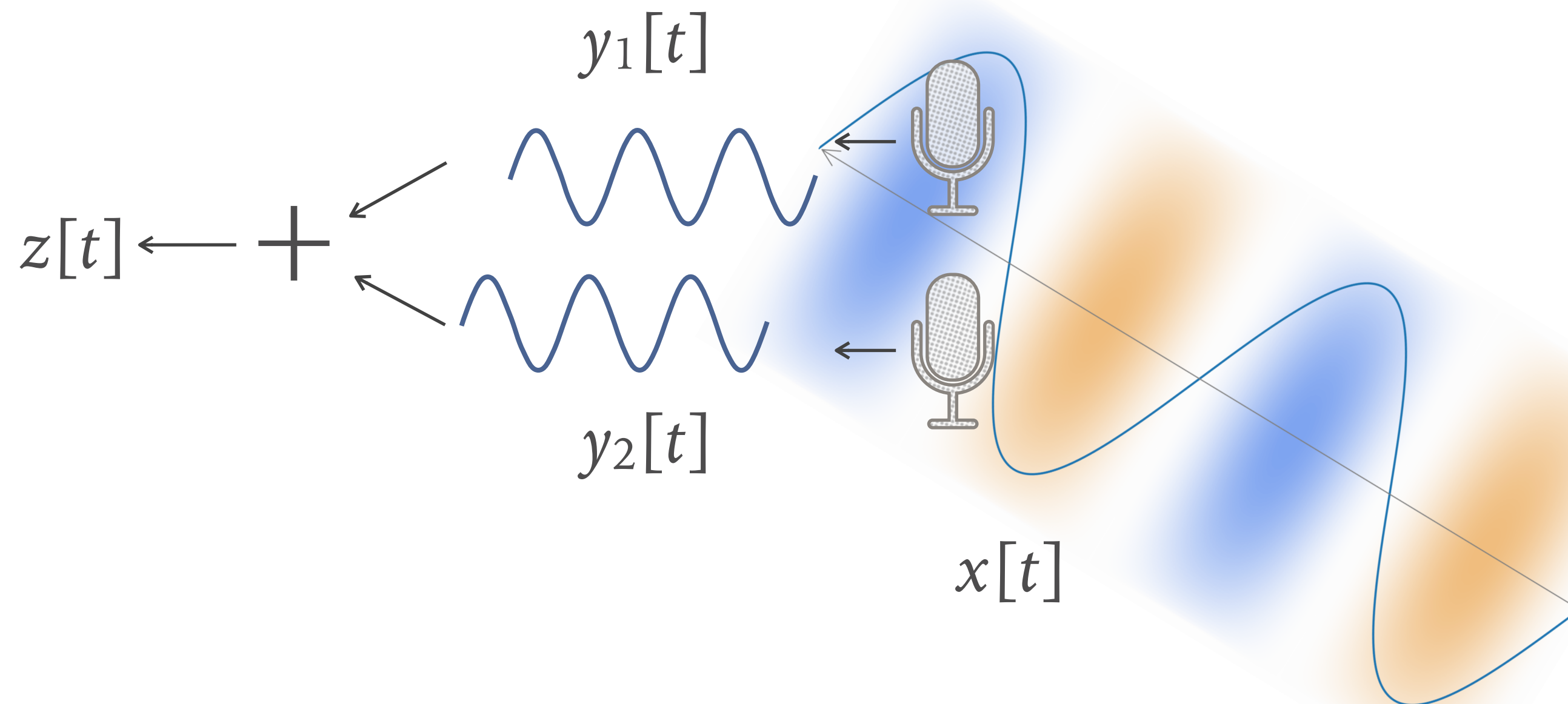
Changing positions

- If the sine comes at an angle it will reach one of the microphones first and the other microphone next
 - The result will be a delay between the mic recordings



Thinking in the frequency domain

- For a sinusoid this delay will be a *phase shift*
 - And we might have a possible attenuation as well
- We can model this with a complex multiplication:



$$x[t] \xrightarrow{DFT} X[k]$$

$$y_i[t] \xrightarrow{DFT} Y_i[k] = v_i^* \cdot X[k]$$

Complex number v_i
scales and phase-shifts
differently for each mic i
(* is conjugation)

To refresh your memory

- Complex number multiplication

$$z = a + \mathbf{i} \cdot b$$

$$w = c + \mathbf{i} \cdot d$$

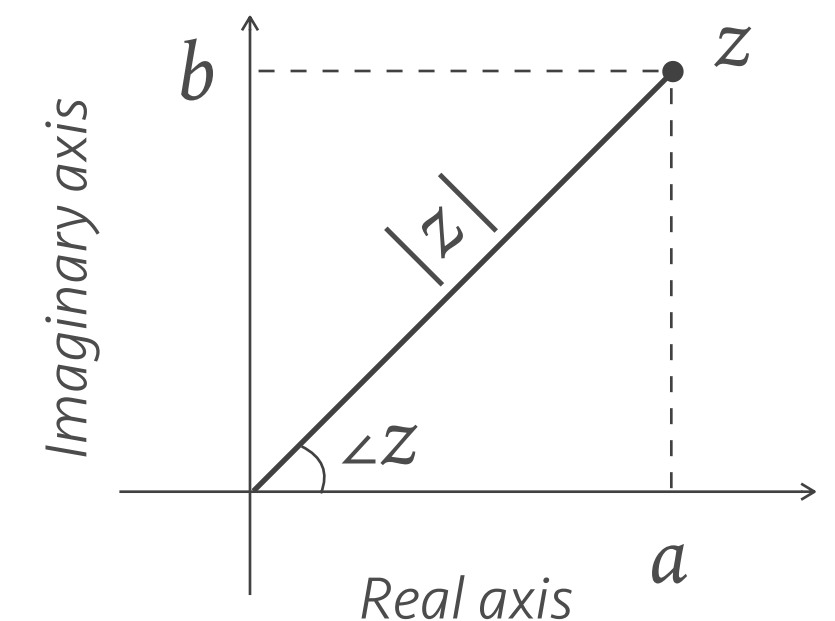
$$z \cdot w = (a + \mathbf{i} \cdot b) \cdot (c + \mathbf{i} \cdot d) = (a \cdot c - b \cdot d) + \mathbf{i} \cdot (a \cdot d + b \cdot c)$$

- In the polar domain

$$z = a + \mathbf{i} \cdot b \xrightarrow{\text{polar}} |z| \cdot \left(\cos(\angle z) + \mathbf{i} \cdot \sin(\angle z) \right)$$

$$w = c + \mathbf{i} \cdot d \xrightarrow{\text{polar}} |w| \cdot \left(\cos(\angle w) + \mathbf{i} \cdot \sin(\angle w) \right)$$

$$z \cdot w = |z| \cdot |w| \cdot \left(\cos(\angle z + \angle w) + \mathbf{i} \cdot \sin(\angle z + \angle w) \right) \quad \text{i.e. magnitudes multiply, phases add}$$



The steering vector

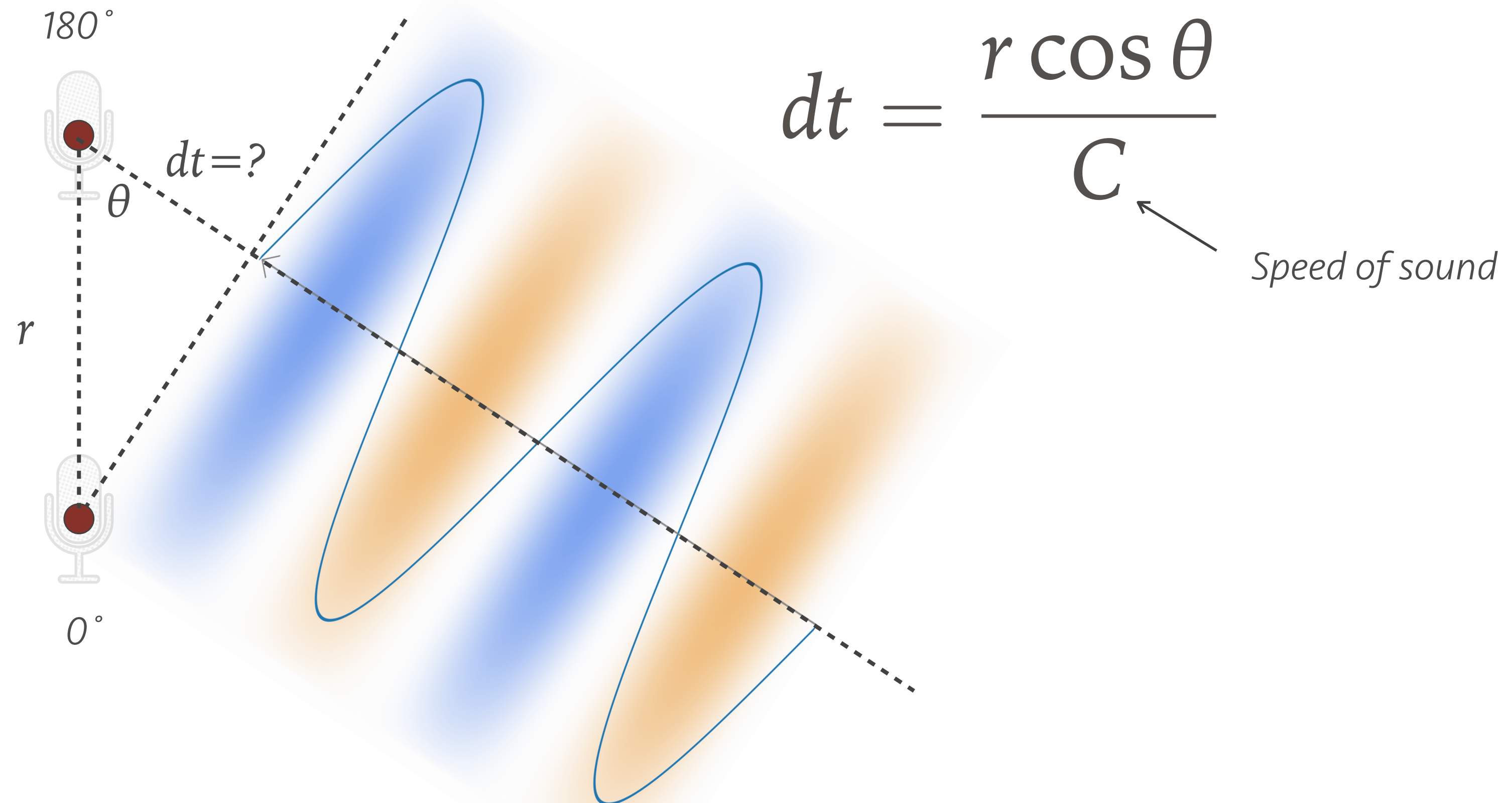
- We associate each direction with a *steering vector*
 - This vector models the sound propagation effects

Sum of shifted inputs $\curvearrowright$ $Z[k] = \sum_i^{\text{\# of mics}} v_i^* X[k] \leftarrow \text{Recording of each mic}$

- And we usually assume that $v_1 = 1$
- But what are the rest of v 's values?

The *far field* model

- Assume that the incoming wave is planar
 - i.e., the sound is so far that the spherical wave curvature is gone



In the digital world

- Angle of incidence $\rightarrow$ sample delay $\rightarrow$ phase shift
 - Will depend on frequency k

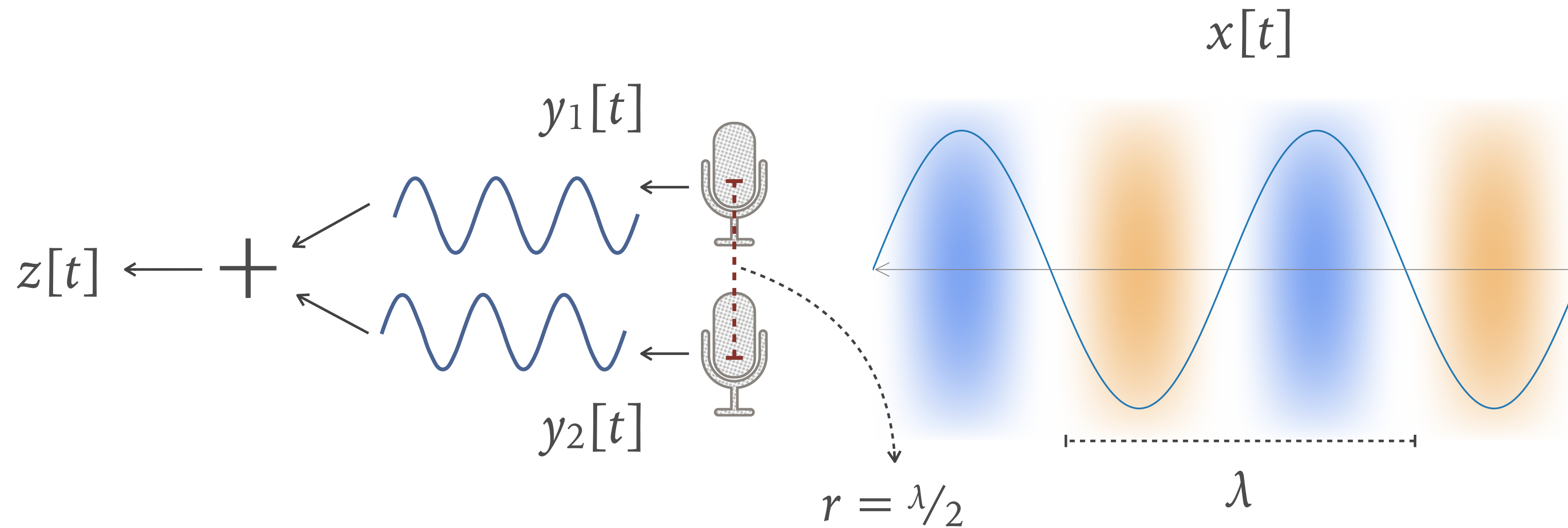
$$\theta \rightarrow d = \frac{r \cos \theta}{C} R \quad \leftarrow \text{Sample rate}$$

$$v_2 = e^{-i d 2\pi \frac{k}{N}}$$

- This is the steering vector for our direction/array
 - Describes delays from each angle θ relative to the array

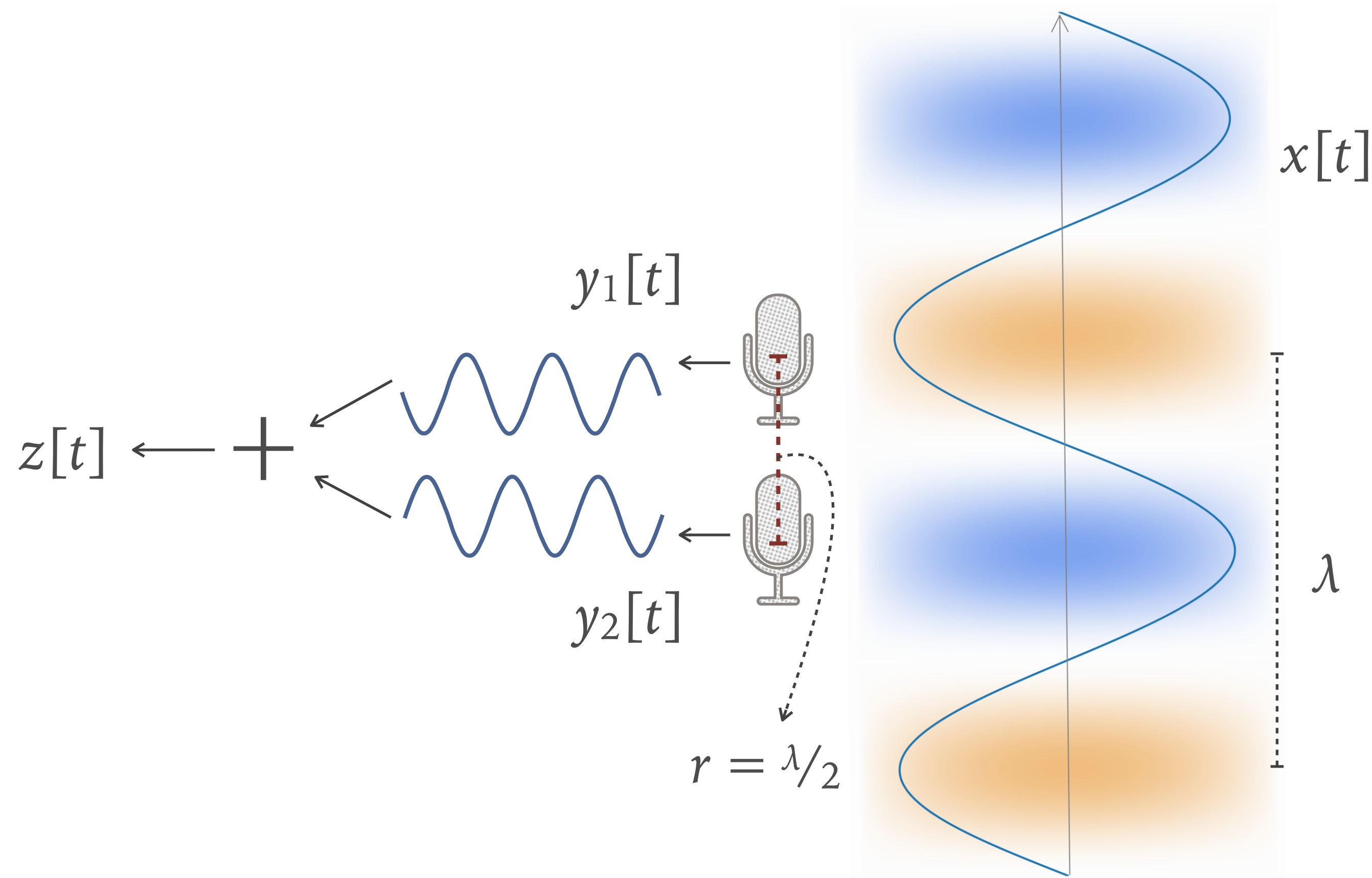
Straight-on case (90°)

- What is the steering vector here?



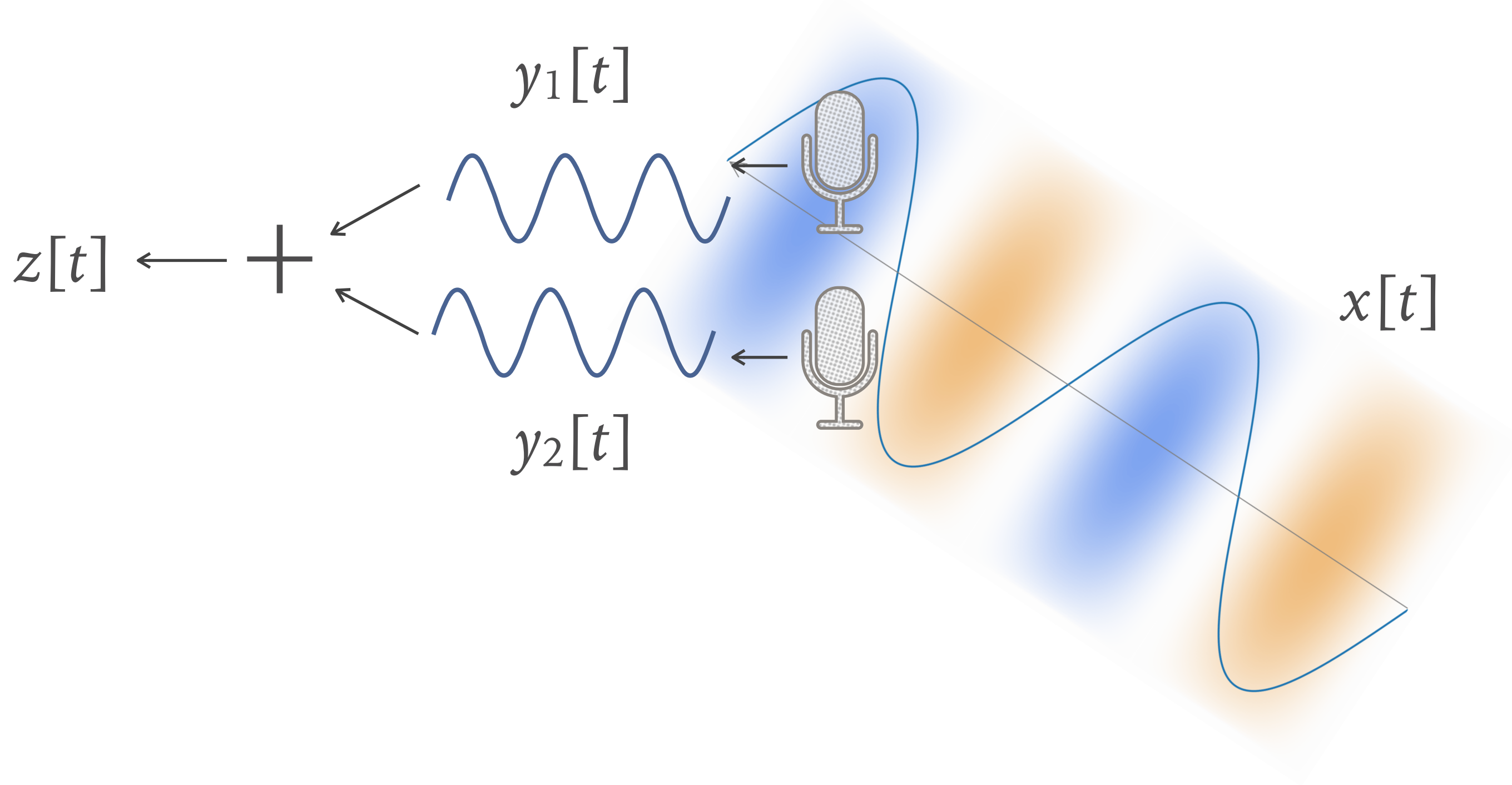
0° direction of incidence

- What will happen now?



Arbitrary direction of incidence

- Cross-mic delay suppresses the input by an arbitrary amount by adding two sines

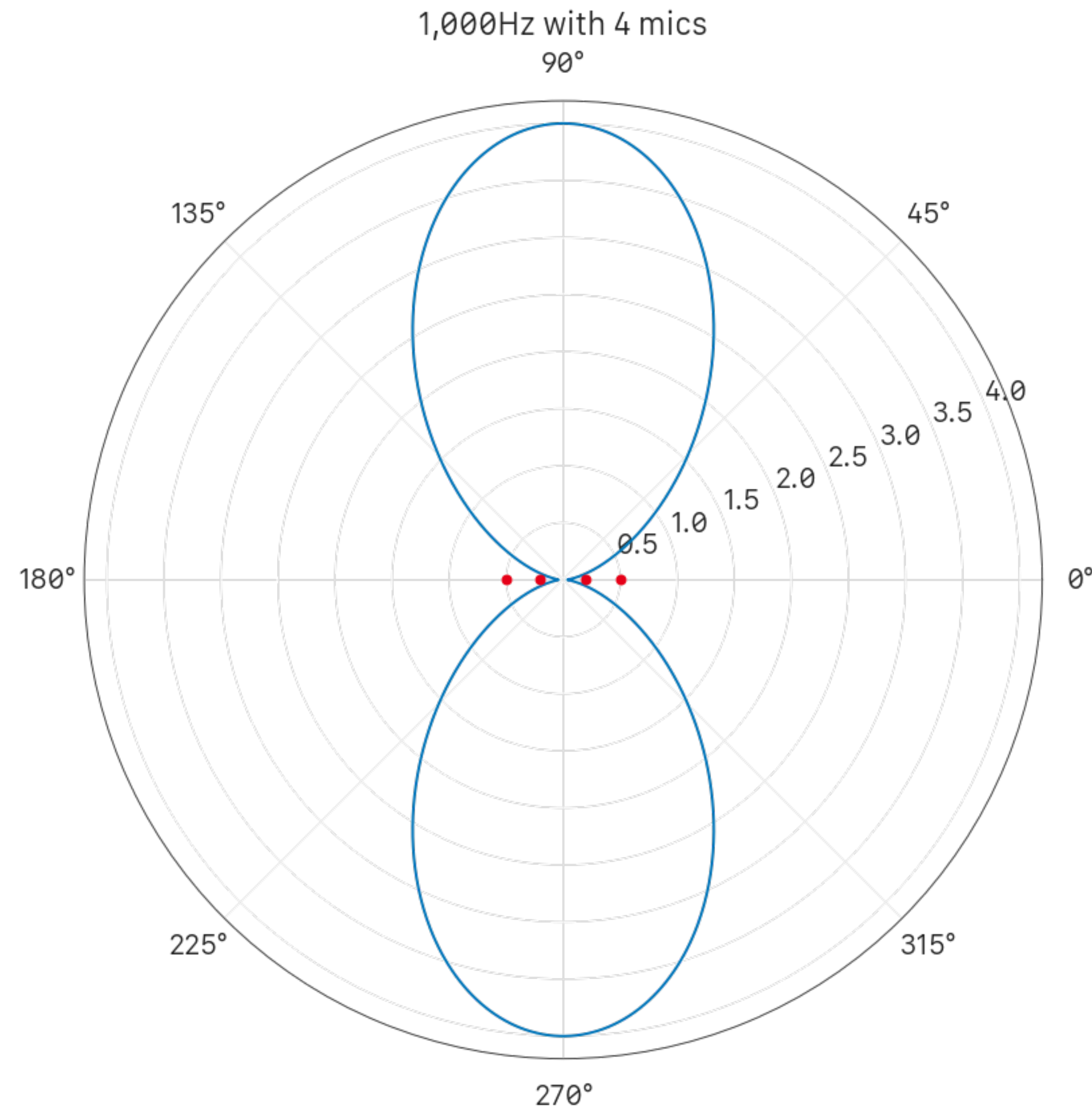


Directional response

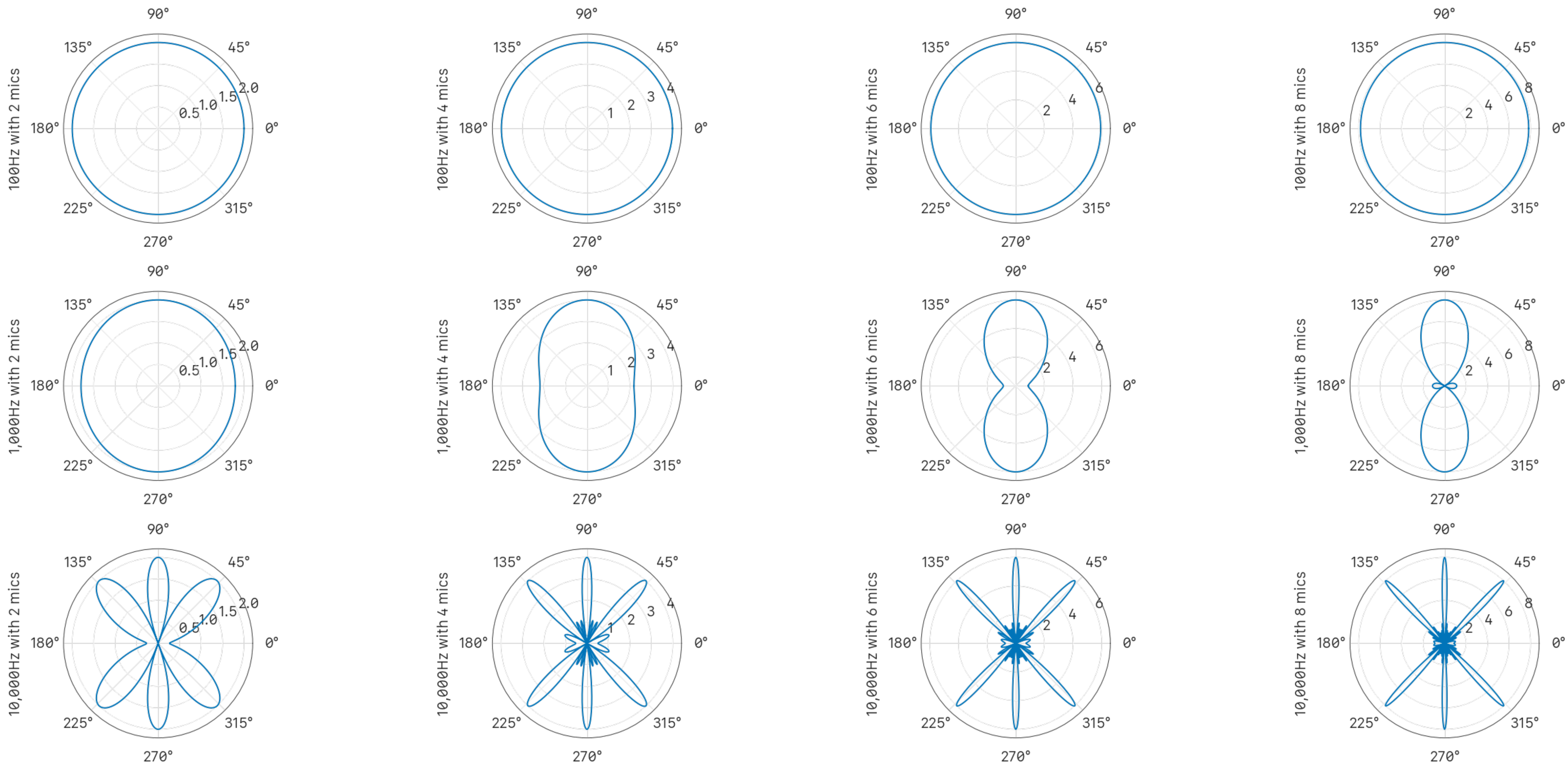
- Mapping the array's response at any angle
 - Let's assume the input is $X[k] = 1$ so that we don't factor that
 - i.e. just measure the steering vectors
- For linear arrays with M mics this becomes:

$$H(k, \theta) = \left| \sum_{m=0}^{M-1} e^{-i 2\pi k \frac{m r \cos \theta}{c}} \right|$$

We visualize these with polar plots



Example responses with $r = 5\text{cm}$



Spatial aliasing

- What was going on in the last row?
- Mic distance needs to be less than half the wavelength of the input signal
 - Otherwise we get *spatial aliasing*
 - We superimpose peaks that are more than one period apart
- This creates *aliased* locations that don't exist
 - Same way that frequency aliasing creates new frequencies

Generalizing to wideband inputs

- Most sounds are not a single sinusoid
- We can add multiple sinusoids with different frequencies/phases/gains to make a broadband sound
- Now we get more steering vectors
 - One set of vectors for each frequency

General form of the steering vector

- Each frequency and angle will have its own steering vector explaining cross-microphone delays

$$v_m(k, \theta) = e^{-j \frac{m r \cos \theta}{c} R \frac{2\pi k}{N}}$$

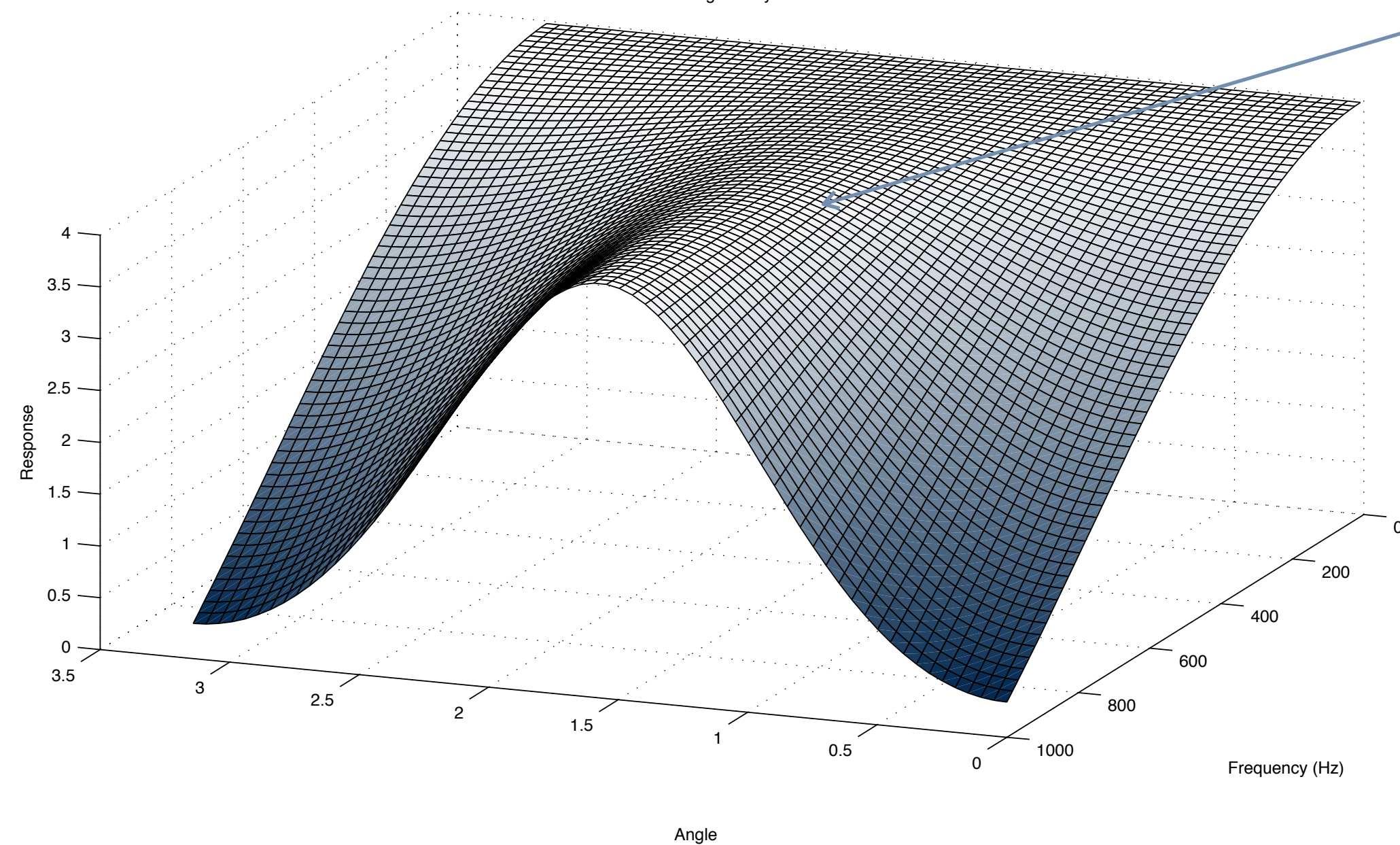
- This formula is for a linear array only
- We can also visualize their responses

Wideband array responses

- Array response is very dependent on the spacing between the mics

8cm array

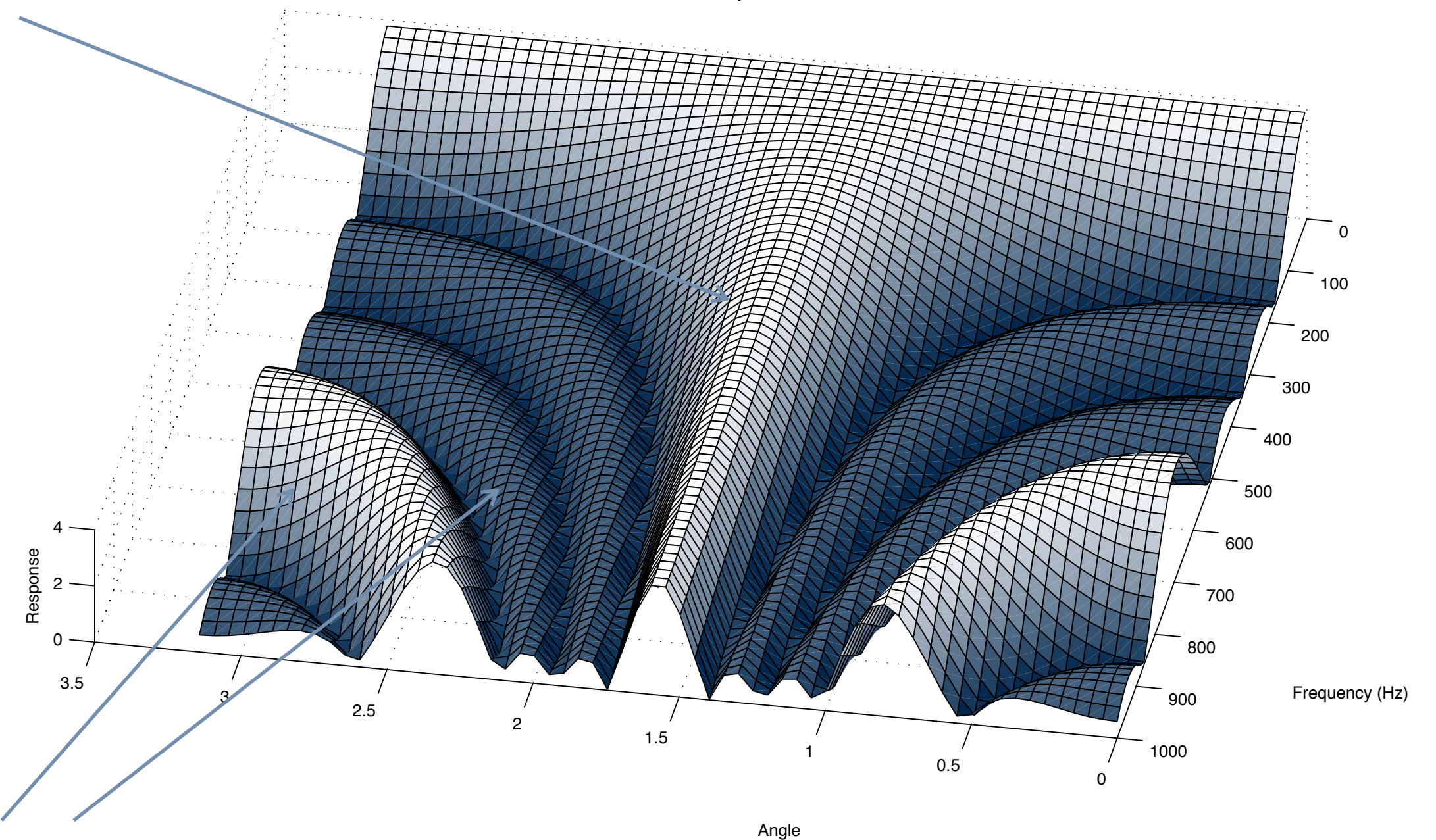
Tight array



Main lobes

50 cm array

Loose array



Grating lobes

How to focus towards one direction

- Undo the effects of the steering vector
 - “Steer” the array towards a location
- Take mic inputs and modulate by:

$$w_m(k, \theta) = e^{j \frac{m r \cos \theta}{c} R \frac{2\pi k}{N}} \leftarrow \begin{array}{l} \text{The original minus sign in} \\ \text{front of } j \text{ is missing, so we are} \\ \text{“undoing” the array’s delay} \end{array}$$

$$Z[k] = \sum w_m(k, \theta) \cdot Y_m[k]$$

- i.e. invert propagation delays so that all microphone inputs superimpose exactly

Delay and sum beamformer

- This is known as the *delay and sum* beamformer
 - The most basic kind of beamformer
- We can now *steer* an array to look towards any angle
 - We simulate “turning” that array towards a direction
- Will boost sounds from the steered direction proportional to number of available microphones
 - i.e. number of times we superimpose

How about suppressing sounds?

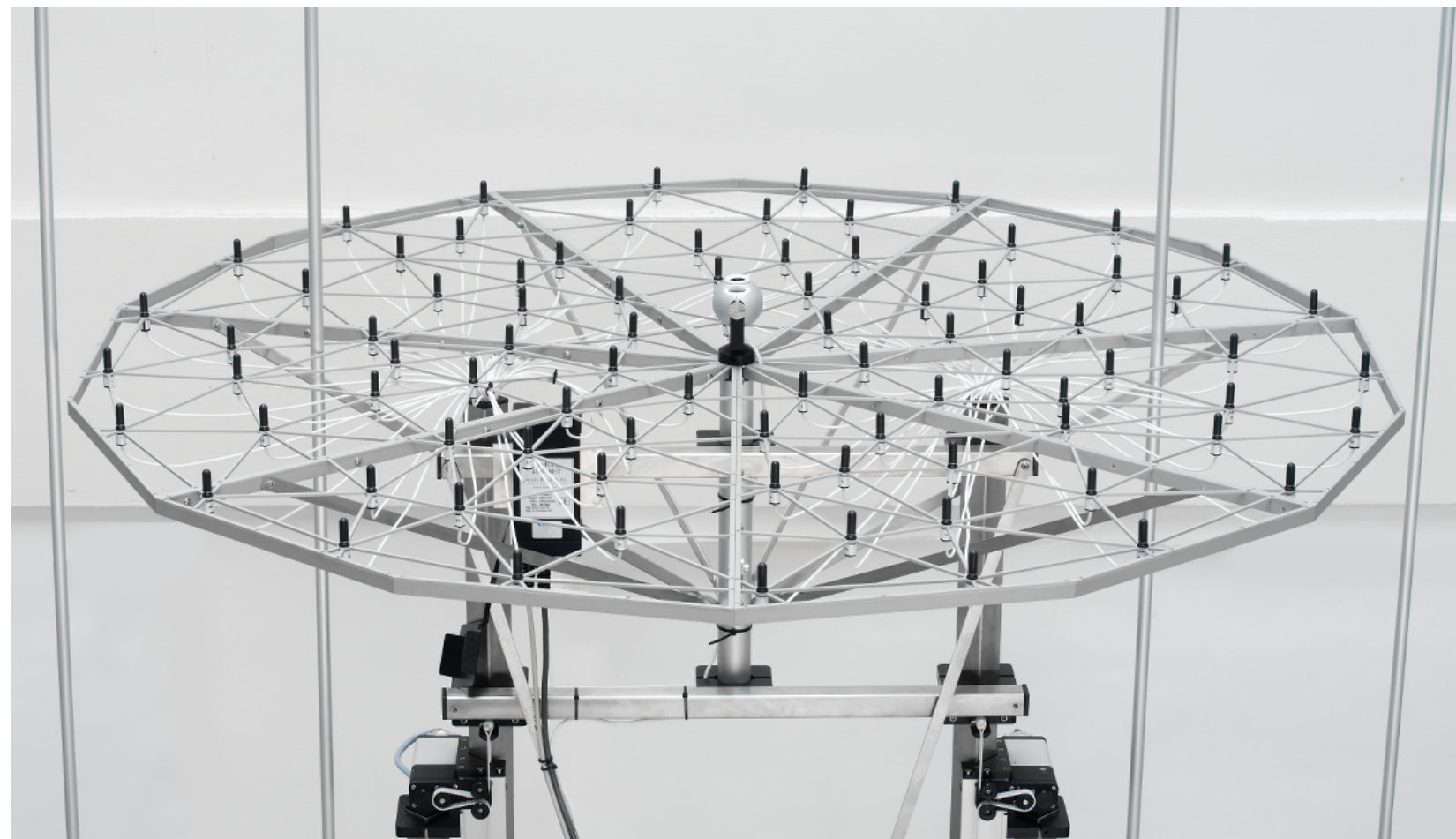
- For now, we steered the array towards a sound to record it louder than the other ones
- We can also manipulate the steering vector to cancel out sounds from a specific direction
 - Thereby removing more of that sound than the others
 - We then create a “null” in space

More advanced types of beamformers

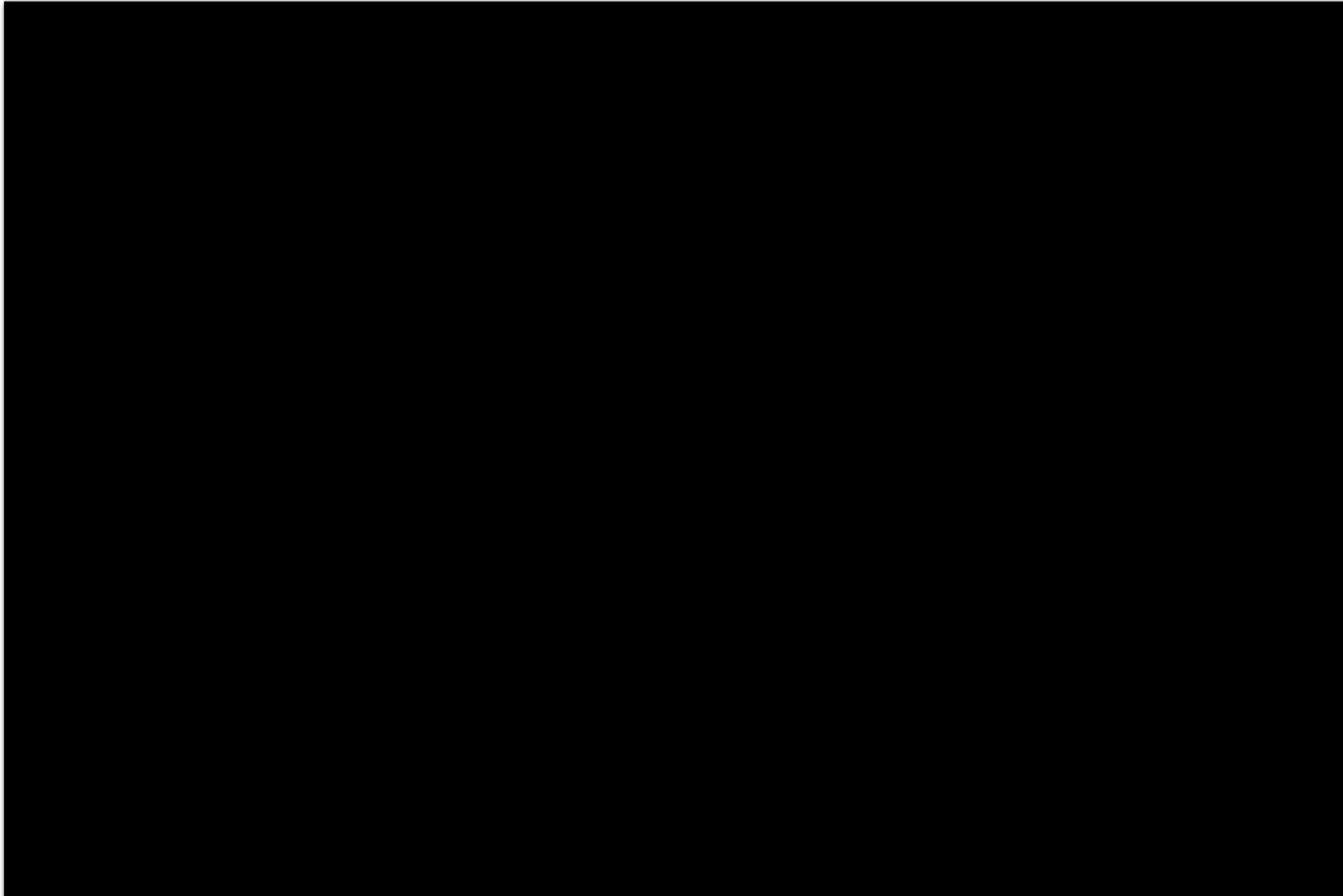
- Minimum Variance Distortionless Response
 - Minimize output variance, but keep gain at 1 for look direction
- Linearly Constrained Minimum Variance
 - Minimize output variance, but specified gains at look directions
- The Generalized Sidelobe Canceller
 - Most common in audio applications
- And many more ...

Other types of arrays

- We only talked about *ULAs*
 - Uniform Linear Arrays
- Other options include spherical, harmonic, logarithmic, grid, or random mic placement



Extreme beamforming at work

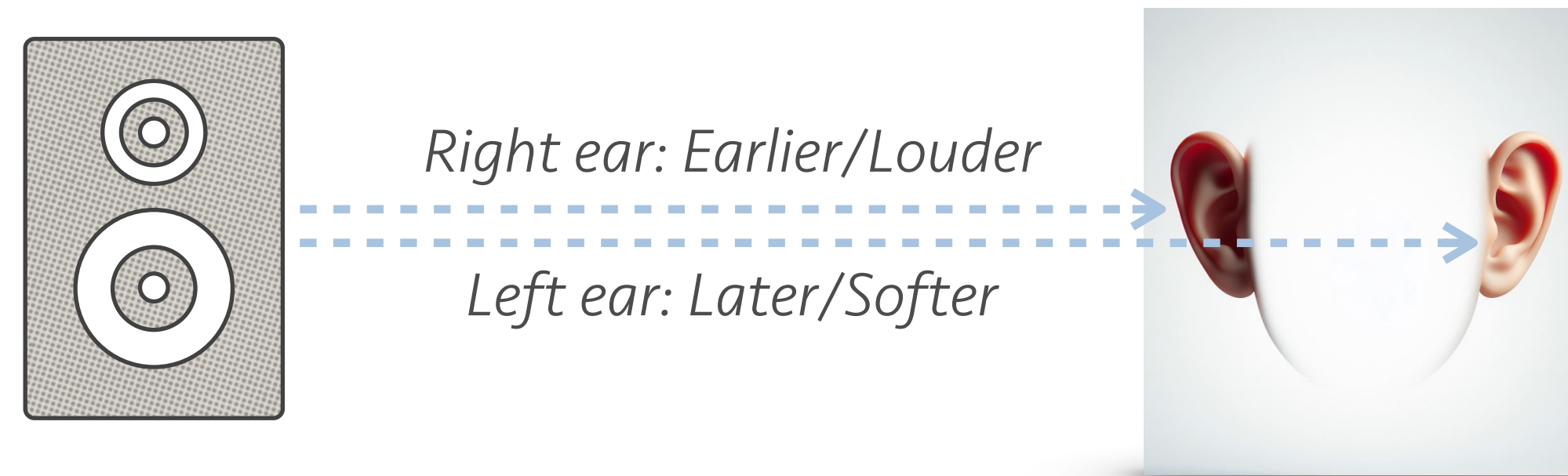


Localization

- The other thing arrays are good for!
- Where is the sound coming from?
 - Why do we care?
 - To steer a beamformer
 - To move teleconference cameras
 - To help robots orient with sound
 - To locate whales (and soviet subs) in the ocean
 - To detect nuclear explosions
 - ...

Why do we have two ears again?

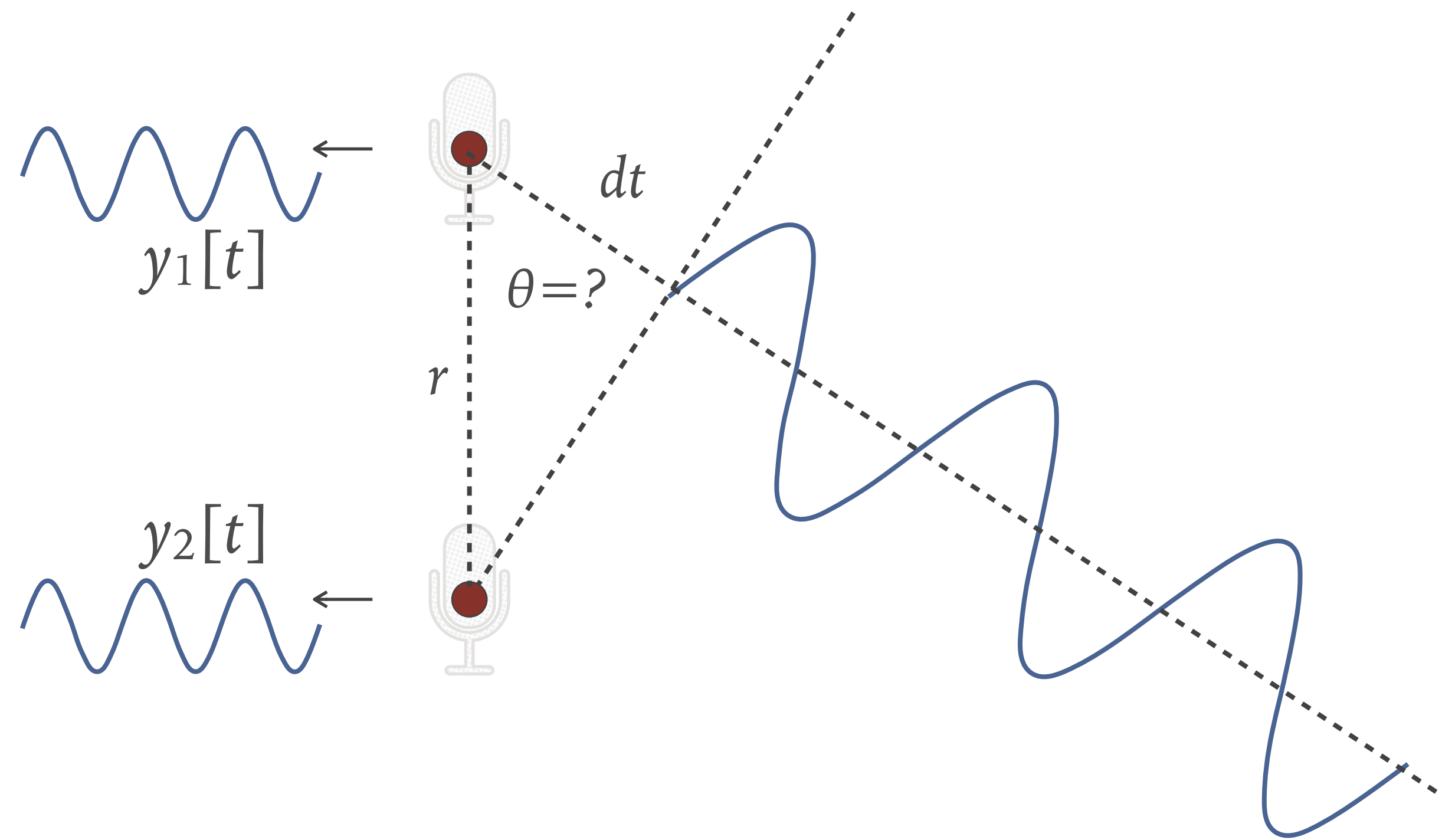
- Using timing and level cues we can pinpoint the direction that a sound is coming from
 - We cannot do that with one ear!



- Likewise, having more than one microphone allows us to *localize* incoming sounds

The far-field model again

- Same problem as before, different unknowns
 - We don't know the angle θ this time



From angle to time-delay

- If we find the sound delay dt between the two mics, we can infer the angle of incidence θ

$$\theta = \cos^{-1} \frac{dt C}{r}$$

- So angle estimation problems are actually time-delay estimation problems

Measuring time delays

- Known as *TDOA* estimation
 - Time Difference Of Arrival
- Basic idea
 - Find how much to delay one of the recordings so that it aligns with the other recording best
 - Note that there will not be an exact match due to noise, etc.

Cross-correlation

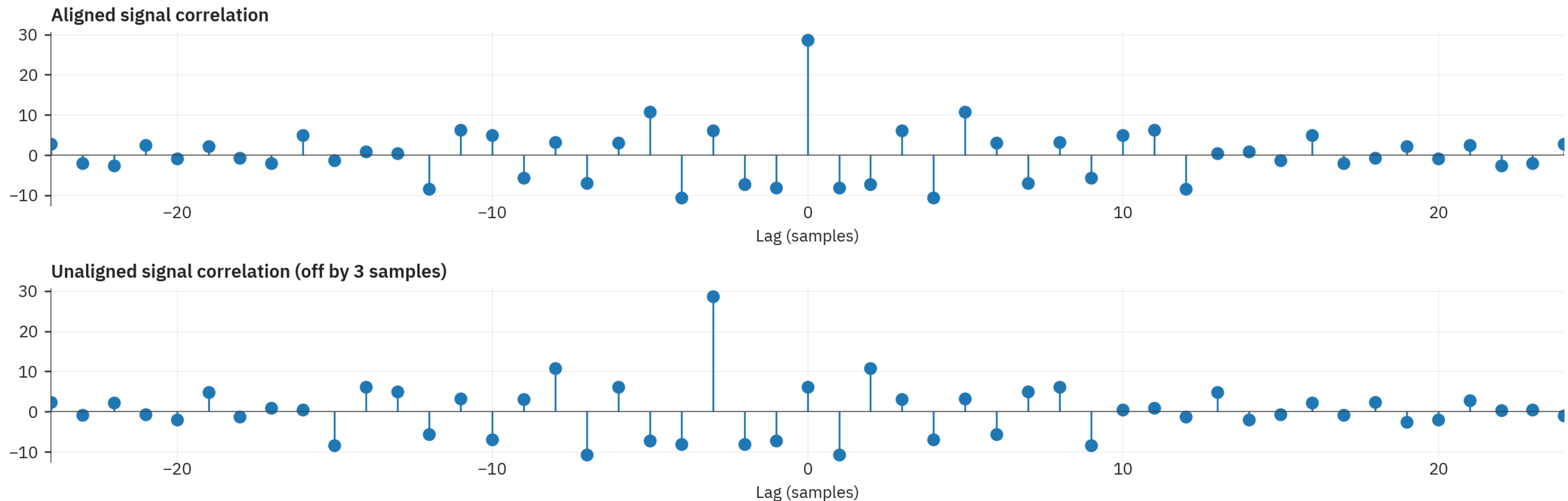
- We can measure signal correlation using:

$$c[d] = \sum_t y_1[t] \cdot y_2[t + d]$$

- This is maximized when y_1 and y_2 are perfectly aligned
- Does this equation look familiar?

Finding a TDOA with cross-correlation

- Perform cross-correlation of two signals and locate the peak of the resulting output



The frequency domain again ...

- It turns out it is more efficient to do this with FFTs
 - Take the FFT of both signals:

$$X_i[k, t] = \text{FFT} \left(x_i[t, \dots, t + N] \right)$$

- Compute cross-correlation:

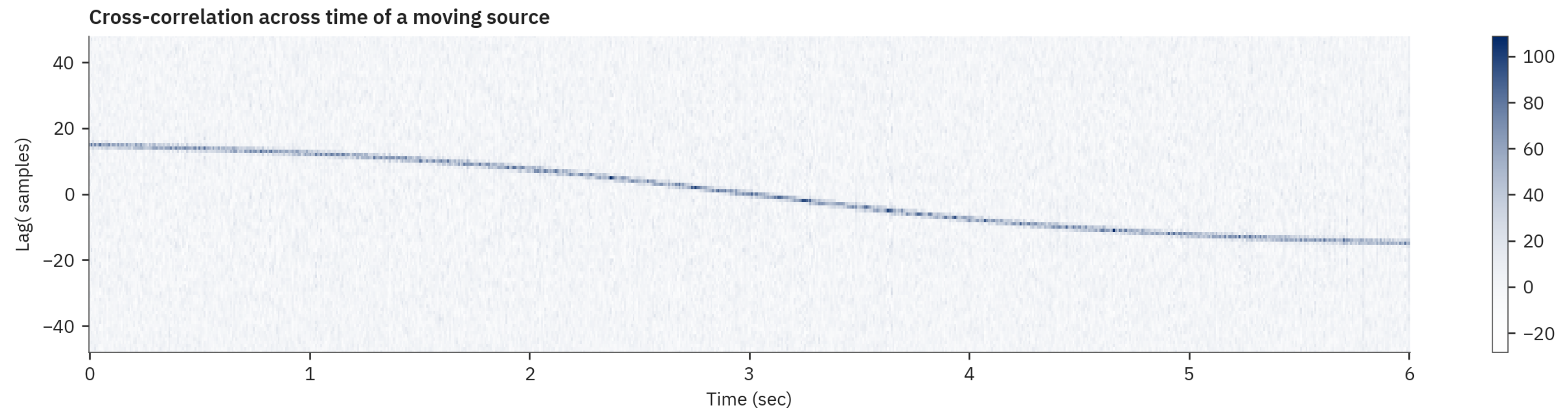
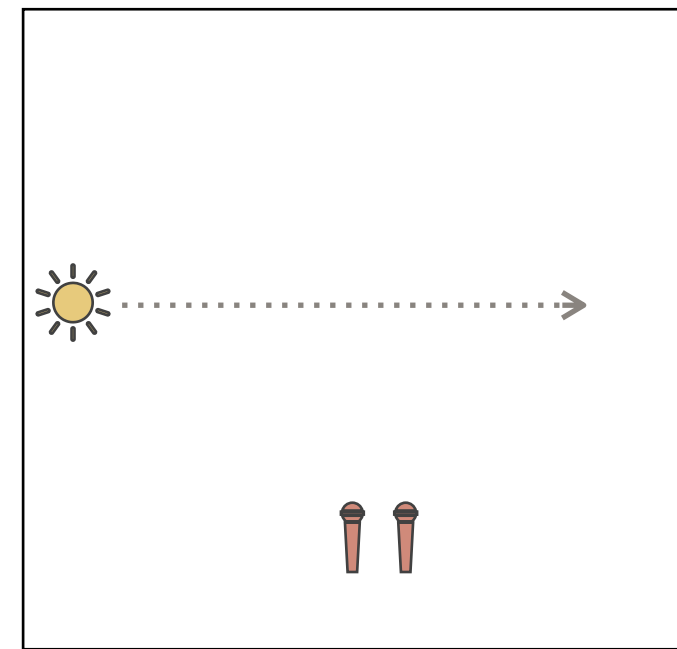
$$Z[k, t] = X_1[k, t] X_2^*[k, t]$$

- Go back to time domain and get the peak:

$$d_t = \arg \max_n z_t[n]$$

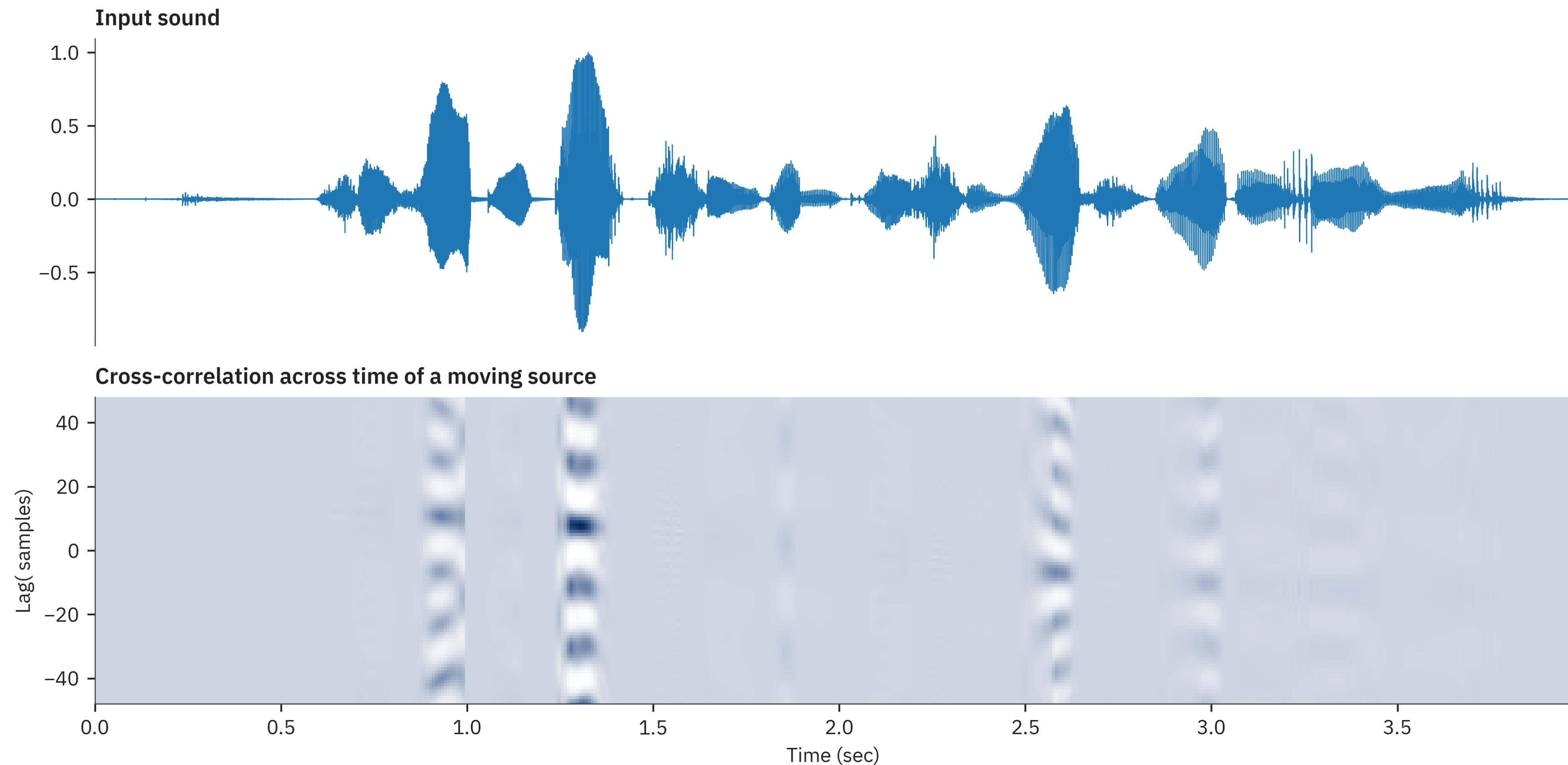
A simple example

- Get inputs, chop to frames, get FFTs, cross-correlate respective frames and show the resulting input



A problem

- Correlation output is dependent on input loudness!



The GCC-PHAT algorithm

- Perform FFT on frames of both input channels:

$$X_i[k, t] = \text{FFT} \left(x_i[t, \dots, t + N] \right)$$

- *Whiten* the spectra:

$$\hat{X}_i[k, t] = \frac{X_i[k, t]}{|X_i[k, t]|}$$

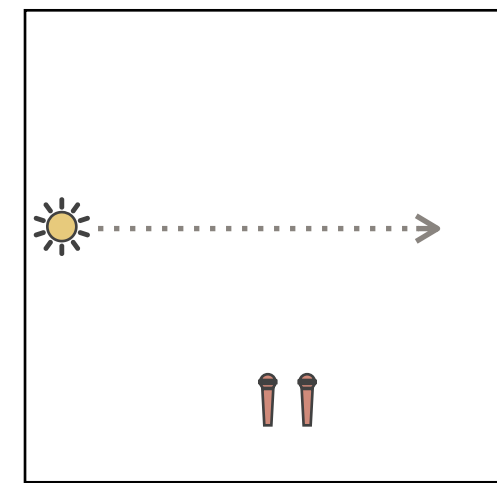
- Compute cross-correlation:

$$Z[k, t] = \hat{X}_1[k, t] \hat{X}_2^*[k, t]$$

- Get peak in the time domain:

$$d_t = \arg \max_n z_t[n]$$

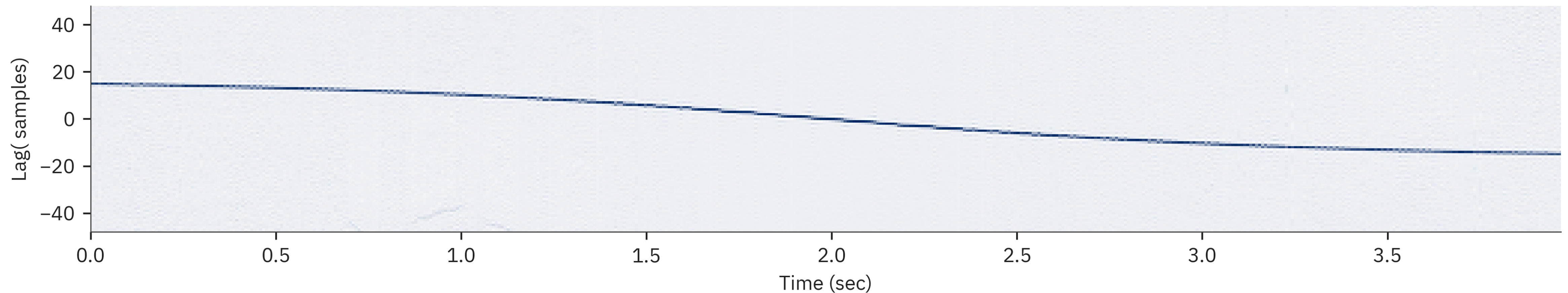
Which looks much better!



Input sound

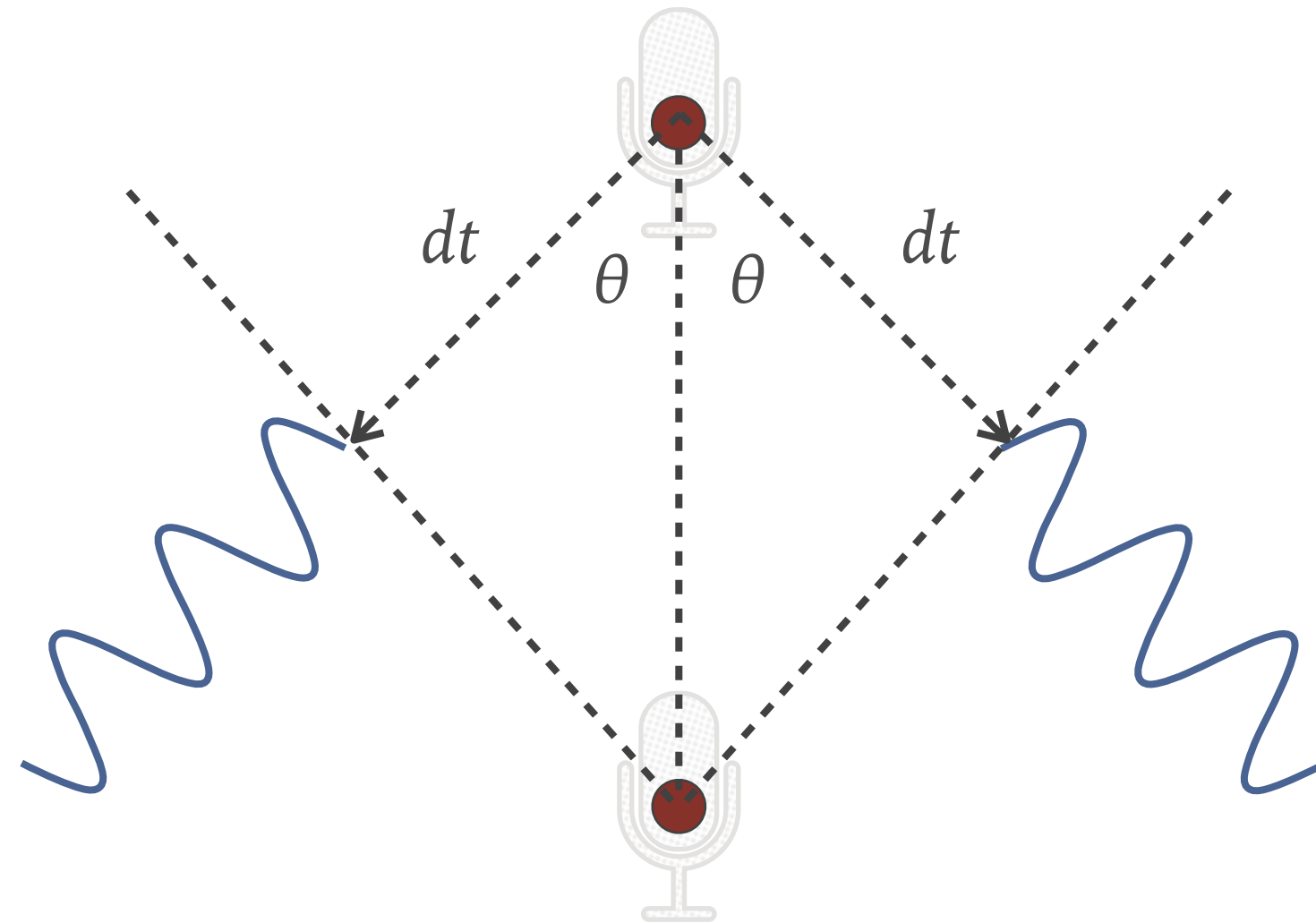


Cross-correlation across time of a moving source



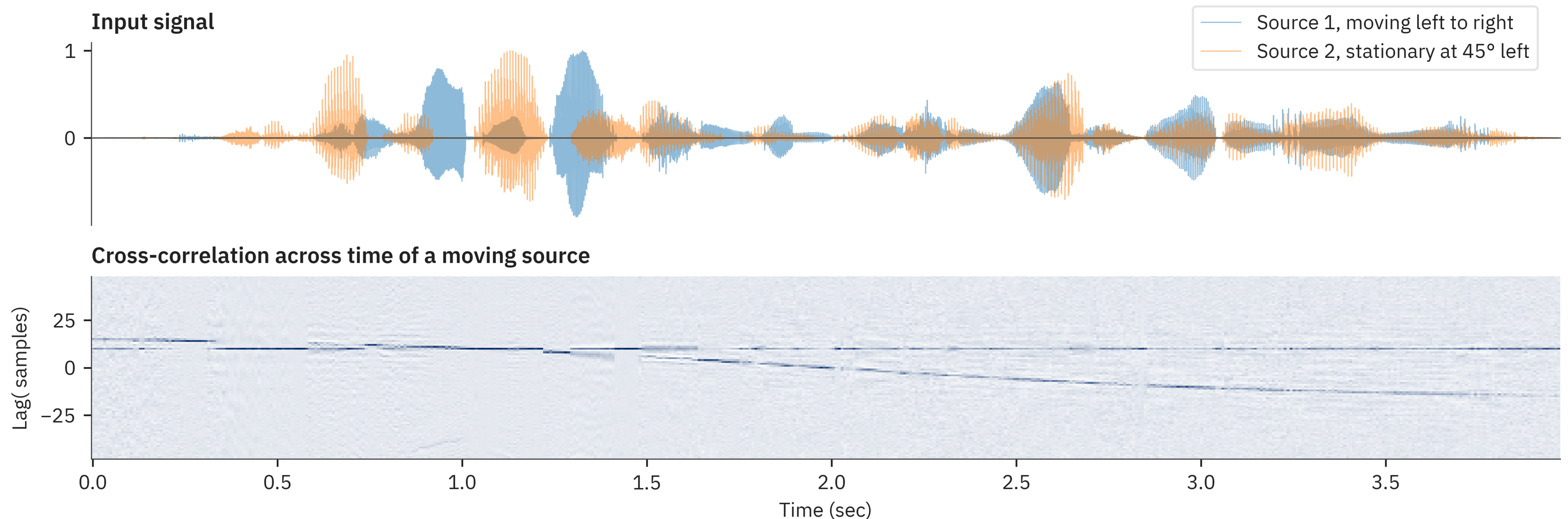
Localization ambiguities

- With two mics you get front/back ambiguity
 - You need at least four mics for 3D localization
 - And the more the better!



Problems with multiple sources

- GCC-PHAT will not work with multiple sources
 - Each location will have its own delay, which do we use?
 - How do we count how many peaks there are?

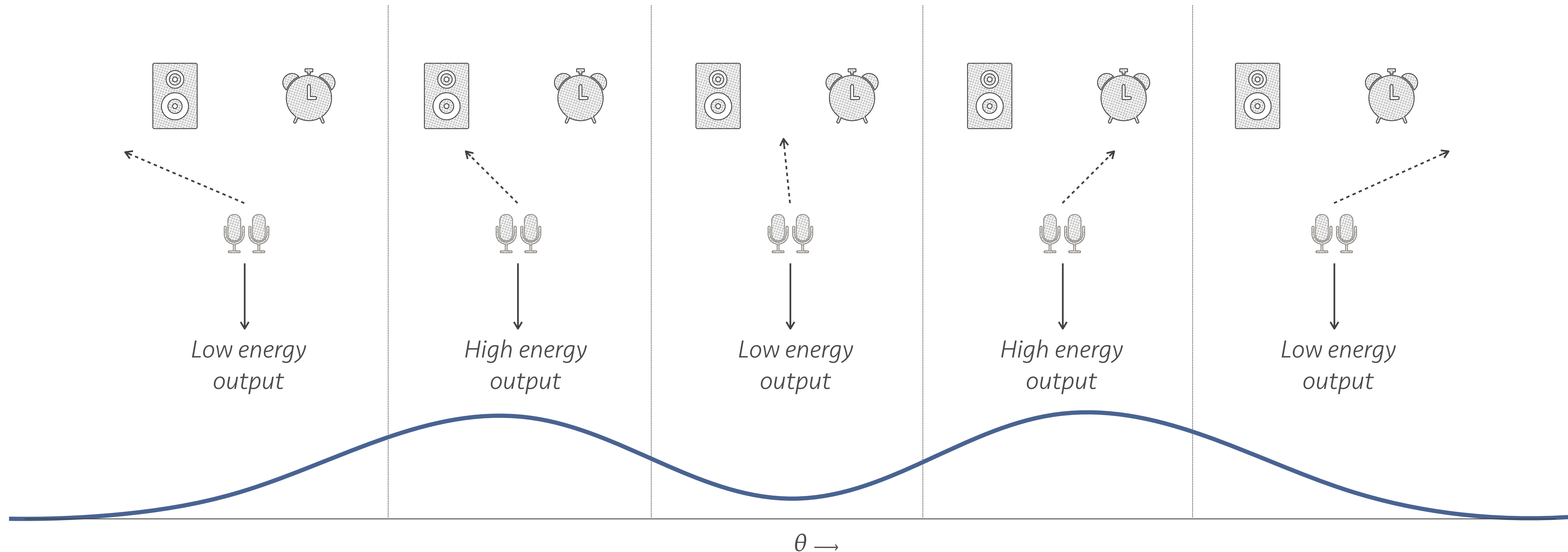


Using a beamformer

- We can instead measure the amount of acoustic energy around our microphone array
 - How much energy is coming from angle θ ?
- To do that we need a beamformer!
 - Steer all around and measure energy from each location
 - Build a histogram of energy over all angles

Mapping acoustic energy

- Steer a beamformer over all directions
 - Measure the energy of each beamformed output

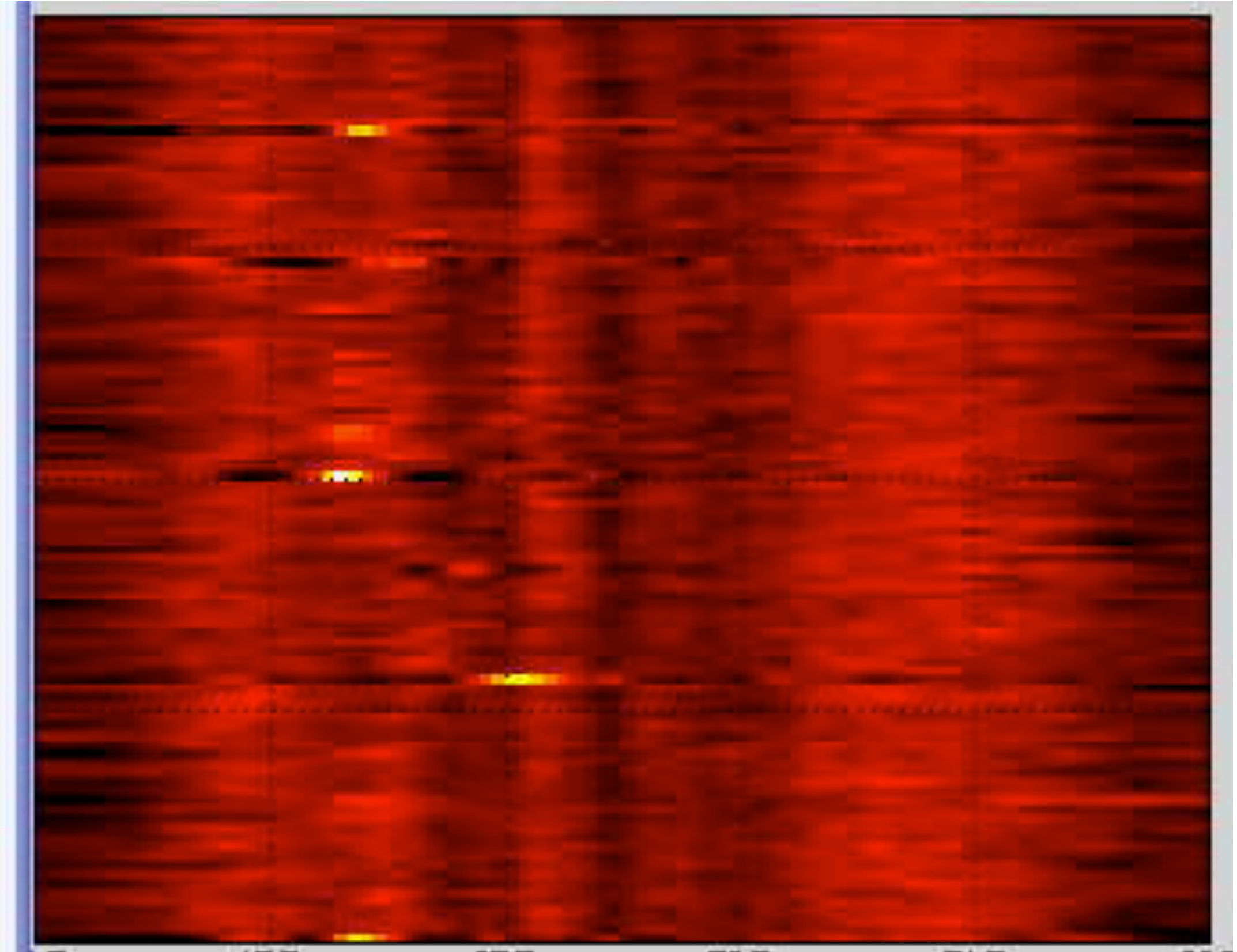


Example of energy mapping

Camera view



Energy over angle and time



Time

Angle

Many more options again

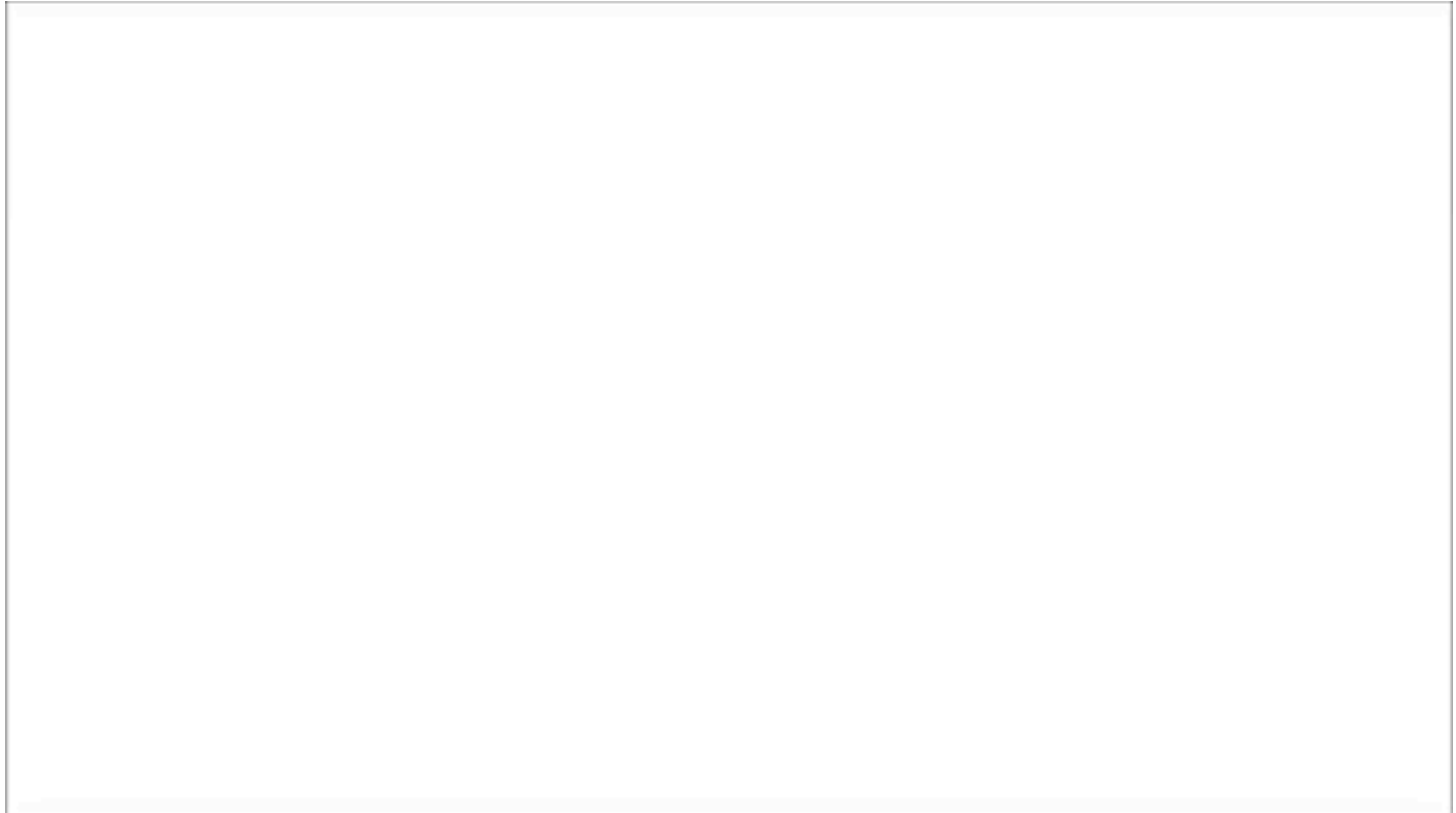
- Time-delay estimation is very well-studied
 - Generalized Cross Correlation (GCC)
 - Multiple Signal Classification (MUSIC)
 - Eigenvalue decompositions
 - Estimation of Signal Parameters via Rotational Invariance Techniques (ESPRIT)
 - ...

Localization at work

- Helping cameras look around



Acoustic imaging



Recap

- **Microphone arrays**
 - Definition, descriptions, etc.
- **Beamforming**
 - Basic array principles
 - Delay and sum beamformer
- **Localization**
 - GCC-PHAT for single source localization
 - Using beamforming for multiple sources

Reference material

- Sound Capture and Processing
 - <https://ieeexplore.ieee.org/book/8039632>
 - Free access from within the UIUC network
- Microphone Array Tutorial
 - http://www.aplu.ch/home/download/microphone_array.pdf
- GCC-PHAT
 - <https://ieeexplore.ieee.org/document/1162830>
 - Free from within UIUC network

Next lab

- What else? Basic array processing!
 - Beamform using delay and sum
 - Localize using cross-correlations